

P2956R0

Allow `std::simd` overloads for saturating operations

Published Proposal, 2023-08-01

This version:

<http://wg21.link/P2956>

Authors:

[Daniel Towner](#) (Intel)

[Ruslan Arutyunyan](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Proposal to extend `std::simd` with overloads for saturating operations

Table of Contents

1 Motivation

2 Wording

2.1 Add new section [`simd.numeric`]

References

Informative References

§ 1. Motivation

[P1928R4] introduced data parallel types to C++. It mostly provided operators which worked on or with `std::simd` types, but it also included overloads of useful functions from other parts of C++ (e.g., `sin`, `cos`, `abs`). Furthermore, in [P0543R2] a proposal is made to provide saturating operation support for some basic arithmetic operations and casts. In particular, `add_sat`, `sub_sat`, `mul_sat` and `div_sat` are provided. These perform saturating arithmetic operations which are effectively performed in infinite precision, and will return the smallest or largest value when it is too large to be represented in that type. In addition, `saturate_cast` is also provided to convert to a new type, and to saturate to the range of that type if required.

These saturating functions should be provided in `std::simd` as element-wise operations.

§ 2. Wording

Below, substitute the  character with a number the editor finds appropriate for the table, paragraph, section or sub-section.

§ 2.1. Add new section [**simd.numeric**]

◆ **basic_simd** numeric library [**simd.numeric**]

```
template<typename T, typename Abi>
constexpr basic_simd<T, Abi> add_sat(const basic_simd<T, Abi>& x,
                                     const basic_simd<T, Abi>& y) noexcept

template<typename T, typename Abi>
constexpr basic_simd<T, Abi> sub_sat(const basic_simd<T, Abi>& x,
                                     const basic_simd<T, Abi>& y) noexcept

template<typename T, typename Abi>
constexpr basic_simd<T, Abi> mul_sat(const basic_simd<T, Abi>& x,
                                     const basic_simd<T, Abi>& y) noexcept

template<typename T, typename Abi>
constexpr basic_simd<T, Abi> div_sat(const basic_simd<T, Abi>& x,
                                     const basic_simd<T, Abi>& y) noexcept
```

Constraints:

- T is a signed or unsigned integral.
- Any constraints that apply to T for the equivalent scalar operation will also apply here.

Preconditions:

- Any preconditions that apply to T for the equivalent scalar operation will also apply here.

Returns:

- A `basic_simd` value with the same type or size, where every value in the range `[0..size)` is formed by performing the given operation on the i^{th} values of the input. If the result is not representable, either the largest or smallest representable value of type T, whichever is closer the actual result of that operation, will be returned.

Remarks:

- The arithmetic operation is performed as a mathematical operation with infinite range and then it is determined whether the mathematical result fits into the result type.

```
template<typename T, typename Abi, typename U>  
constexpr simd<T, basic_simd<U, Abi>::size()>  
saturate_cast(const basic_simd<U, Abi>& v) noexcept;
```

Constraints:

- T is a signed or unsigned integral.

Returns:

- A `basic_simd` value with the same size as the input, but of the new element type U. For every element of the return value, the i^{th} element will be a copy of the i^{th} element of v, otherwise it will be the largest or smallest value of type U, whichever is closer to the original value of that element.

§ References

§ Informative References

[P0543R2]

Jens Maurer. *Saturation arithmetic*. 18 September 2022. URL: <https://wg21.link/p0543r2>

[P1928R4]

Matthias Kretz. *std::simd - Merge data-parallel types from the Parallelism TS 2*. 19 May 2023. URL: <https://wg21.link/p1928r4>